

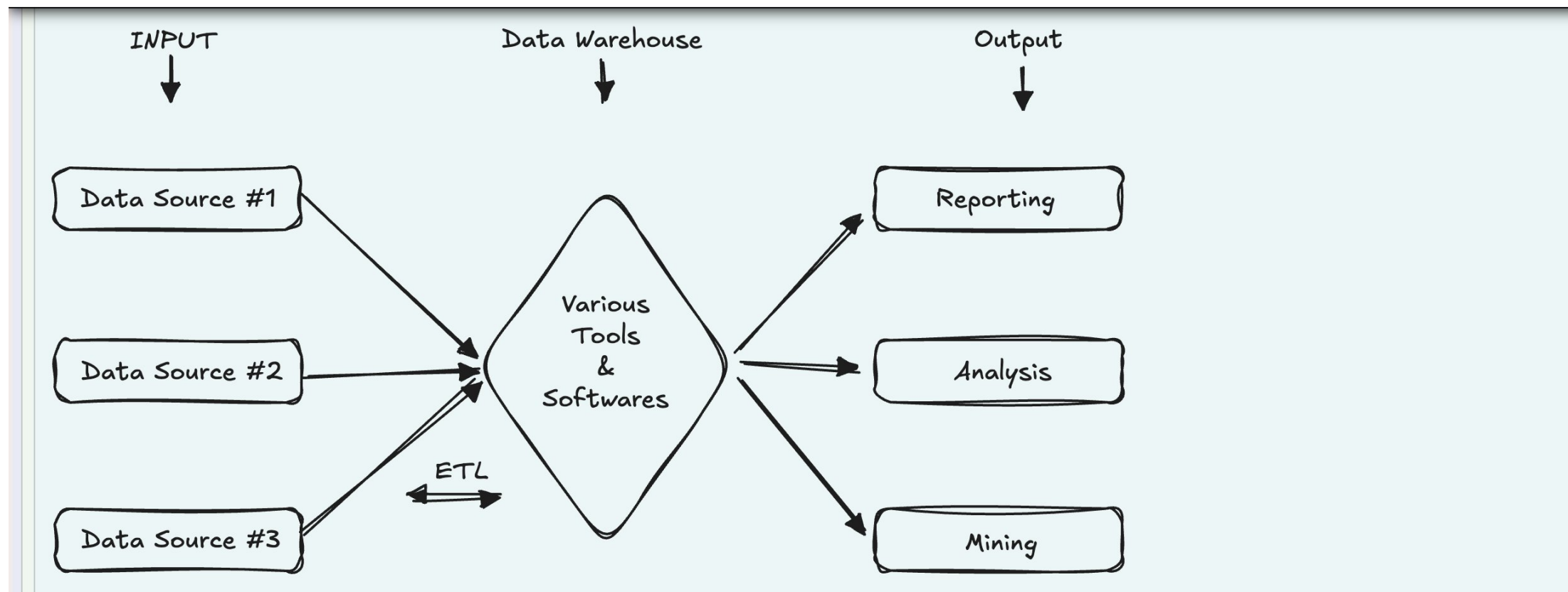
Chapter -1

Data Mining

- **Data Warehouse Introduction**

- Imagine you run a big retail chain like Amazon. You have data coming in from multiple places; website orders, store purchases, customer service interactions, and even supplier data. Now, if this data is scattered across different systems, how will you analyze trends, predict sales, or make decisions?
- That's where a Data Warehouse (DW) comes in. It acts like a central storage hub, where data from different sources is collected, cleaned, and structured for better reporting and analysis.
- A Data Warehouse (DW) is a centralized storage system that integrates data from multiple sources, making it available for reporting, analysis, and decision-making. Unlike regular databases, a data warehouse is optimized for query performance, historical data storage, and business intelligence.
- Examples of Data Warehouses
 - Amazon Redshift
 - Google BigQuery
 - Snowflake
 - Microsoft Azure Synapse

How a Data Warehouse Works



- **Step 1: Data Sources (Input)**
- Data comes from multiple sources like:
 - Sales databases
 - Customer interactions
 - Social media insights
 - Supplier information
- **Step 2: Data Warehouse (Processing)**
- This is where the magic happens! The data is organized, processed, and stored using various tools and software.
- **Step 3: Output (Usage of Data)**
- Once stored, business use the data for:
 - Reporting (e.g., daily sales reports)
 - Analysis (e.g., why are sales dropping in a certain region?)
 - Data Mining (e.g., finding hidden patterns in customer behavior)

- **Real-Life Example**

- Let's say McDonald's wants to track its daily sales across thousands of outlets worldwide.
- Without a data warehouse, they would have to check each outlet's database separately, which would be a nightmare! Instead, they collect all sales data, process it in a central system, and analyze it to make business decisions.
- For example, they can use this data to:
 - Identify which menu items sell the most in different regions
 - Forecast demand for ingredients to avoid shortages
 - Optimize pricing based on sales trends

The Role of ETL in a Data Warehouse

- Before the data gets stored, it needs to be processed properly. That's where ETL (Extract, Transform, Load) comes in.
 - Extract – Data is pulled from different sources (e.g., POS systems, customer apps).
 - Transform – Data is cleaned, formatted, and structured. (E.g., dates might be stored differently in different systems, so they need to be standardized.)
 - Load – The processed data is stored in the data warehouse, ready for analysis.

Why Do We Even Need a Data Warehouse?

- Imagine you own a large retail business with stores in multiple cities. Every day, your stores generate huge amounts of data—sales transactions, customer purchases, stock levels, and more. All this data is stored in traditional databases, known as Database Management Systems or DBMS. These systems are designed to help with storing, updating, and retrieving current data efficiently, mainly for daily operations and transactions.
- But here's the problem. What if you want to analyze your data to make better business decisions? For example, you might want to know:
 - What was the total sales revenue last year?
 - Which city has the highest demand for electronics?
 - How do this year's sales compare to last year's?

- These questions aren't about day-to-day operations; they're about trends, summaries, and comparisons over time. That's what we call analytical processing, and it's a different task altogether from just managing current transactions.
- A regular DBMS isn't built for these kinds of tasks. It excels at handling real-time transactional data but struggles with large-scale, historical analysis. That's where the concept of a data warehouse comes in—a specialized system designed to store and organize massive amounts of historical data from various sources. Unlike DBMS, a data warehouse is optimized for complex queries, reporting, and data analysis, using what's known as Online Analytical Processing (OLAP), instead of the transaction-focused Online Transaction Processing (OLTP).

Data warehouse schema

- Data warehouse schema is a description, represented by objects such as tables and indexes, of how data relates logically within a data warehouse.
- Star, Galaxy and Snowflake Schema are types of warehouse schema that describe different logical arrangements of data.
- We can think of dataware house schema as a blueprint or an architecture of how data will be stored and managed. A data warehouse schema isn't the data itself, but the organization of how data is stored and how it relates to other data within the data warehouse .

- The basic components of all data warehouse schemas are fact and dimension tables.

Fact Table

- A fact table aggregates metrics, measurements or facts about business processes.
- In this example, fact tables are connected to dimension tables to form a schema architecture representing how data relates within the data warehouse.
- Fact tables store primary keys of dimension tables as foreign keys within the fact table.

Dimension Table

- Dimension tables are non-denormalized tables used to store data attributes or dimensions.
- The primary key of a dimension table is stored as a foreign key in the fact table.
- Dimension tables are not joined together. Instead, they are joined via association through the central fact table.

Fact Table

Revenue	—————	Dealer_ID
		Model_ID
		Branch_ID
		Data_ID
		Unites_Sold
		Revenue

Dimension Table

Dealer	—————	Dealer_ID
		Location_ID
		Country_ID
		Dealer_NM
		Dealer_CNTCT

Three prominent types of data warehouse schema known as **Star Schema**, **Snowflake Schema** and **Galaxy Schema**.

Each of these data warehouse schemas has unique design constraints and describes a different organizational structure for how data is stored and how it relates to other data within the data warehouse.

What is a Star Schema in a data warehouse?

- The star schema in a data warehouse is historically one of the most straightforward designs. This schema follows some distinct design parameters, such as only permitting one central table and a handful of single-dimension tables joined to the table. In following these design constraints, star schema can resemble a **star with one central table, and five dimension tables joined** (thus where the star schema got its name).
- Star Schema is known to **create denormalized dimension tables—a database structuring strategy that organizes tables to introduce redundancy for improved performance.**
- Denormalization intends to introduce redundancy in additional dimensions so long as it improves query performance.

- In a **denormalized** schema:

Data **redundancy** is **intentionally** introduced.

- This means **repeating data** (e.g., storing a customer's name in multiple rows instead of referencing it from a single place).
- In contrast to normalized designs (used in transactional databases to reduce redundancy), **denormalization** helps in **faster querying**, especially in reporting or analytical use cases.

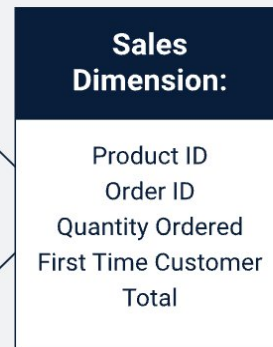
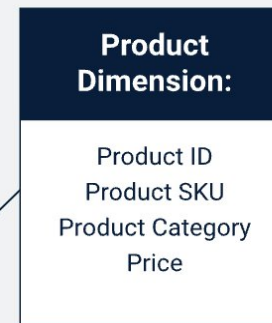
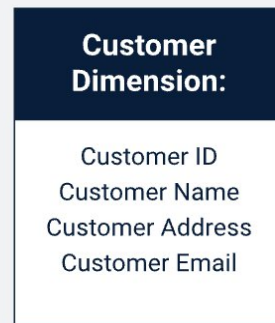
- A **Star Schema** uses **denormalized dimension tables**—meaning it repeats some data in order to **simplify structure** and **speed up queries**. This is acceptable in analytical systems, where **query performance is more important than storage efficiency**.

Example

- Instead of creating multiple normalized tables for Country, City, and Region, a denormalized Location dimension table may have all this data in **one row**:
Location_ID | Country | State | City.

- **Characteristics of the Star Schema:**

- Star data warehouse schemas create a denormalized database that enables quick querying responses
- The primary key in the dimension table is joined to the fact table by the foreign key
- Each dimension in the star schema maps to one dimension table
- Dimension tables within a star scheme are not to be connected directly
- Star schema creates denormalized dimension tables

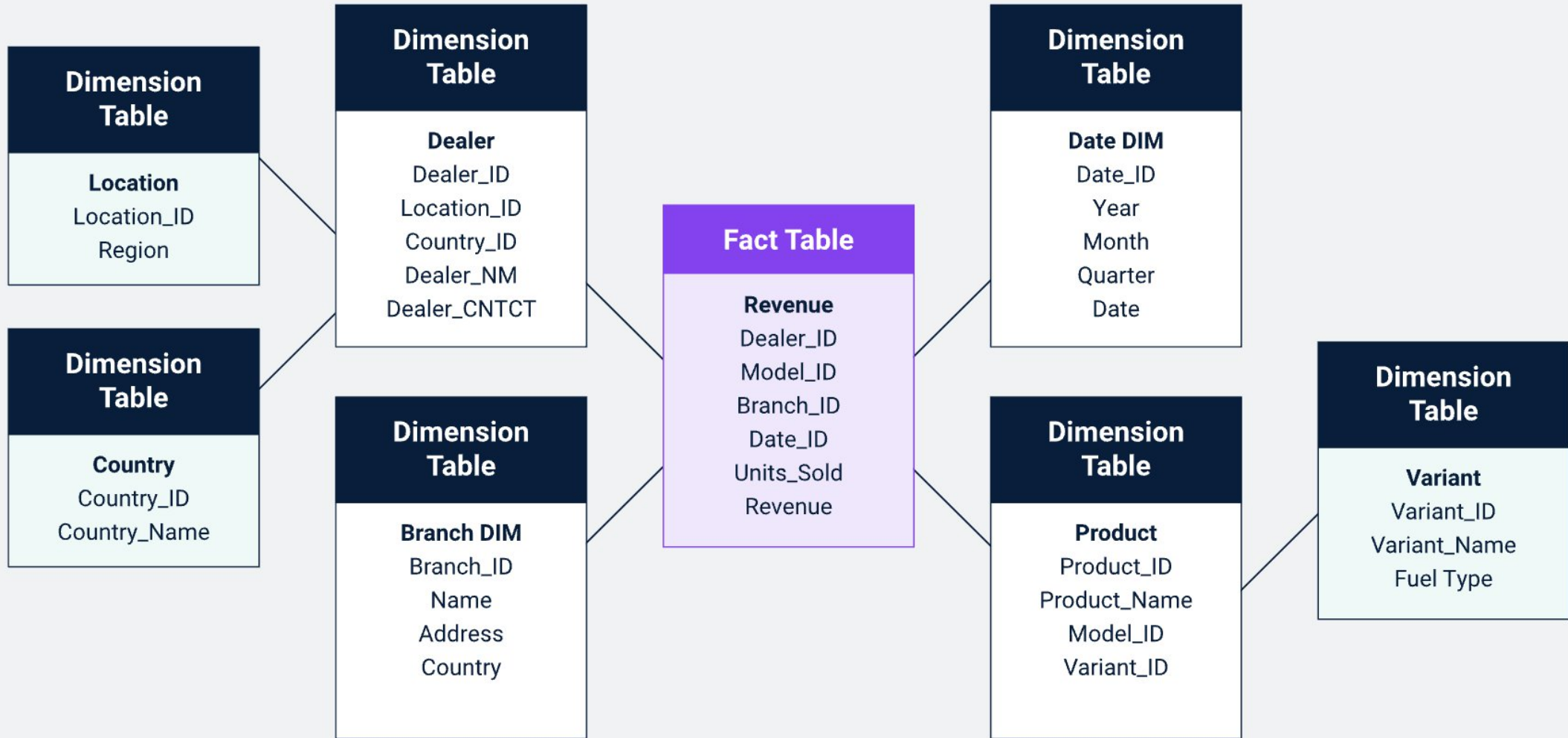


- **What is a Snowflake Schema?**

- The Snowflake Schema is a data warehouse schema that encompasses a logical arrangement of dimension tables. This data warehouse schema builds on the star schema by adding additional sub-dimension tables that relate to first-order dimension tables joined to the fact table.
- Just like the relationship between the foreign key in the fact table and the primary key in the dimension table, with the snowflake schema approach, a primary key in a sub-dimension table will relate to a foreign key within the higher order dimension table.
- Snowflake schema creates normalized dimension tables—a database structuring strategy that organizes tables to reduce redundancy. The purpose of normalization is to eliminate any redundant data to reduce overhead.
-

Characteristics of the Snowflake Schema:

- Snowflake Schema are permitted to have dimension tables joined to other dimension tables
- Snowflake Schema are to have one fact table only
- Snowflake Schema create normalized dimension tables
- The normalized schema reduces required disk space for running and managing this data warehouse
- Snowflake Scheme offer an easier way to implement a dimension



star schema for a retail sales database

star schema for a retail sales database

Fact Table: FactSales

- **SalesID** (Primary Key)
- **DateKey** (FK → Date Dimension)
- **ProductKey** (FK → Product Dimension)
- **CustomerKey** (FK → Customer Dimension)
- **StoreKey** (FK → Store Dimension)
- **SalesAmount**
- **QuantitySold**
- **Discount**
- **Dimension Tables**

1.DimDate

1. DateKey (PK)
2. FullDate
3. Day
4. Month
5. Quarter
6. Year

2.DimProduct

1. ProductKey (PK)
2. ProductName
3. Category
4. Subcategory
5. Brand
6. Price

1. DimCustomer

- 1. CustomerKey (PK)**
- 2. FirstName**
- 3. LastName**
- 4. Gender**
- 5. Age**
- 6. City**
- 7. State**
- 8. Country**

2. DimStore

- 1. StoreKey (PK)**
- 2. StoreName**
- 3. Location**
- 4. Region**
- 5. Manager**

DimDate

|

FactSales

/

/

\

\

DimProduct DimCustomer DimStore

- **OLAP** stands for ***Online Analytical Processing*** Server.
- It is a software technology that allows users to analyze information from multiple database systems at the same time.
- OLAP cubes support powerful operations to analyze and explore data from different perspectives.
- It is based on multidimensional data model and allows the user to query on multi-dimensional data (eg. Delhi -> 2018 -> Sales data). OLAP databases are divided into one or more cubes and these cubes are known as *Hyper-cubes*.

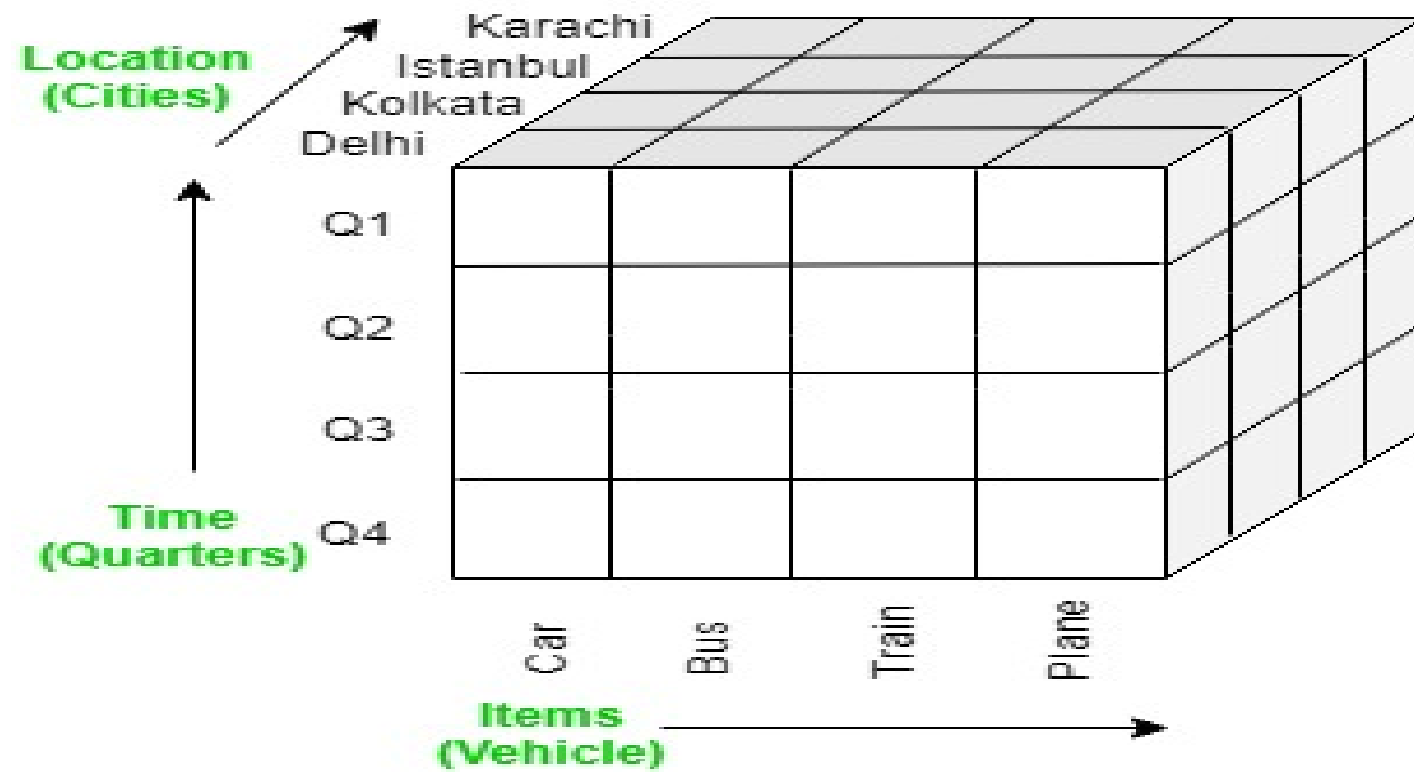
- **Online Analytical Processing (OLAP)**

- Online Analytical Processing (OLAP) refers to software tools used for the analysis of data in business decision-making processes.
- OLAP systems generally allow users to extract and view data from various perspectives, many times they do this in a multidimensional format which is necessary for understanding complex interrelations in the data. These systems are part of data warehousing and business intelligence, enabling users to do things like trend analysis, financial forecasting, and any other form of in-depth data analysis.

- **OLAP Examples**

- Any type of Data Warehouse System is an OLAP system. The uses of the OLAP System are described below.
- Spotify personalizes homepages with custom songs and playlists based on user preferences.
- Netflix movie recommendation system.

- **Multidimensional Data Model:**
- Data is stored and viewed in the form of a **cube** (also called a **hyper-cube** when more than 3 dimensions).
- Each dimension could be something like:
 - **Time** (Year → Quarter → Month),
 - **Location** (Country → State → City),
 - **Product**, etc.
- Example: You can analyze **Sales** for **Delhi** in **2018**, broken down by **Product Categories**.
- **Cubes (Hyper-cubes):**
- OLAP organizes data into **cubes** that allow fast access to data across various dimensions.
- Each cube contains **measures** (numeric data like sales, profit) and **dimensions** (categories for analysis).
- **Fast Query Performance:**
- OLAP systems are optimized for **read-heavy** operations like reporting, dashboards, and analytics.
- **Complex Calculations:**
- Supports functions like aggregation (sum, average), comparisons (YoY growth), slicing, dicing, drilling down, etc.



- **OLAP operations:**

1. There are five basic analytical operations that can be performed on an OLAP cube:

Drill down: In drill-down operation, the less detailed data is converted into highly detailed data. It can be done by:

1. Moving down in the concept hierarchy
2. Adding a new dimension

In the cube given in overview section, the drill down operation is performed by moving down in the concept hierarchy of *Time* dimension (Quarter -> Month).

- **Drill Down**

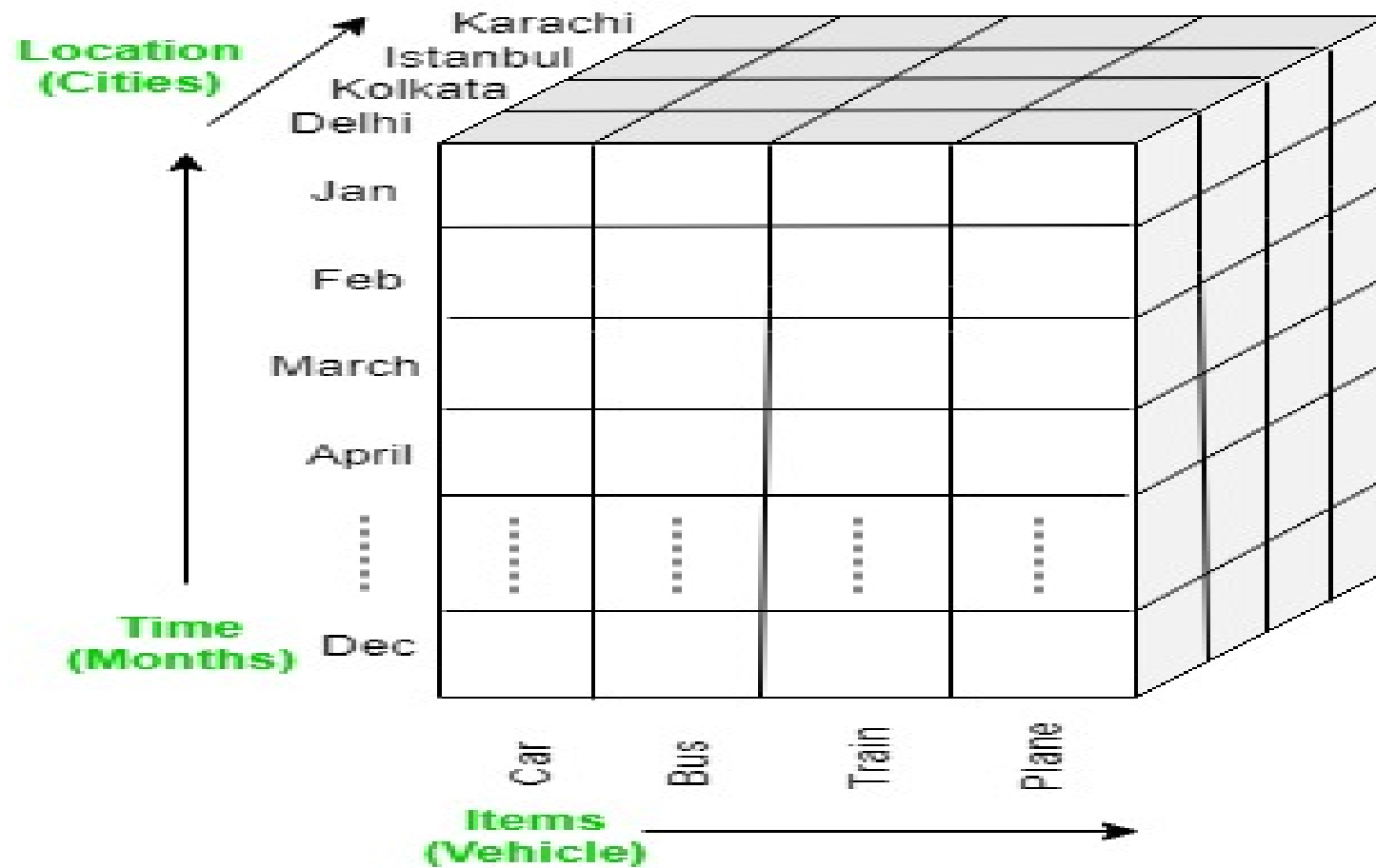
To view **more detailed data**.

- **How it works:**

- Move **down the hierarchy** in a dimension.
 - Example: Time → from **Year** → **Quarter** → **Month** → **Day**
- OR add a **new dimension** for deeper analysis.

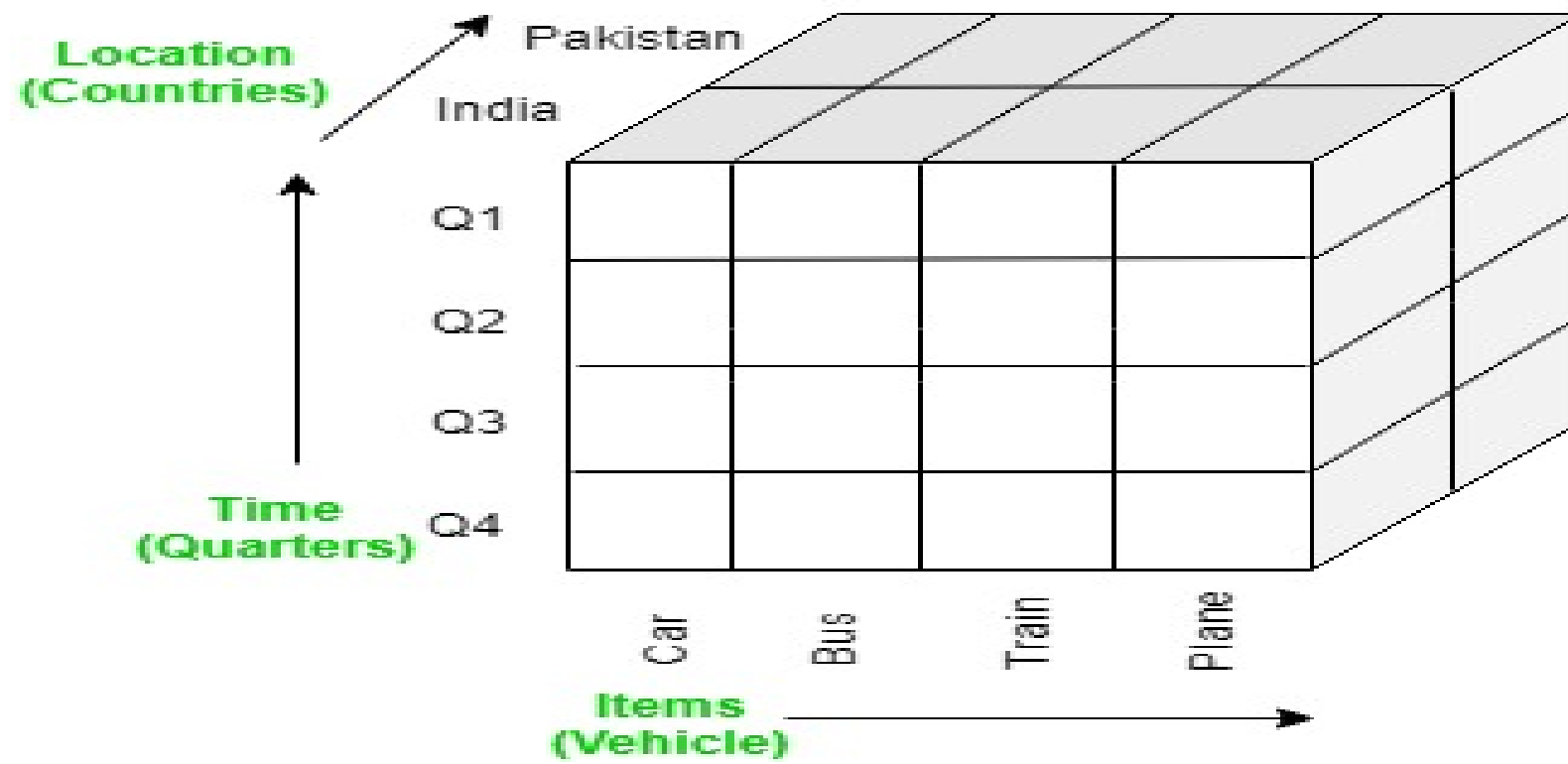
- **Example:**

- Original view: Sales in **Q1 2024**
- Drill down: Sales in **January, February, March 2024**



- **Roll up:** It is just opposite of the drill-down operation. It performs aggregation on the OLAP cube. It can be done by: Climbing up in the concept hierarchy
- Reducing the dimensions
- In the cube given in the overview section, the roll-up operation is performed by climbing up in the concept hierarchy of *Location* dimension (City -> Country).

- **Roll Up**
- To view **less detailed or more summarized data** (opposite of drill-down).
- **How it works:**
 - Move **up the hierarchy**.
 - Example: City → State → Country
 - OR remove a dimension.
- **Example:**
 - Original: Sales in **Delhi, Mumbai, Bangalore**
 - Roll-up: Sales in **India**



- **Dice:** It selects a sub-cube from the OLAP cube by selecting two or more dimensions. In the cube given in the overview section, a sub-cube is selected by selecting following dimensions with criteria: Location = "Delhi" or "Kolkata"
- Time = "Q1" or "Q2"
- Item = "Car" or "Bus"

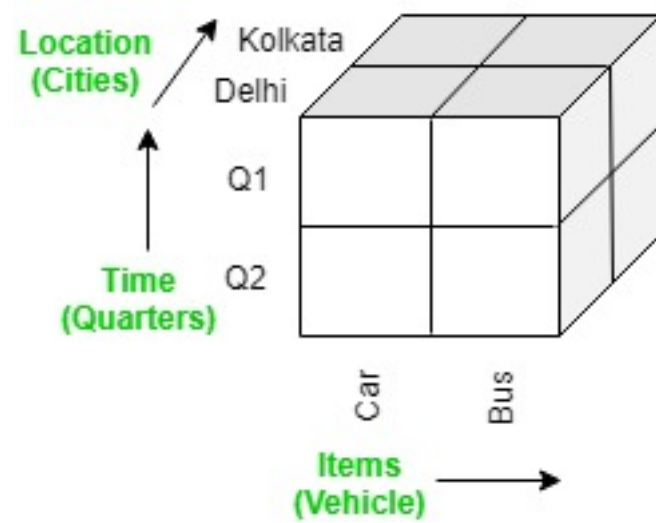
Slice

•**Purpose:** To view a **subset** of the cube by fixing a value in one dimension.

•**Example:**

- Fix "Year = 2023"
- View sales across **all locations** and **products** for 2023

Like cutting a single layer (or "slice") of the cube.



- **Slice:** It selects a single dimension from the OLAP cube which results in a new sub-cube creation. In the cube given in the overview section, Slice is performed on the dimension Time = "Q1".

Location (Cities)

Karachi				
Istanbul				
Kolkata				
Delhi				
	Car	Bus	Train	Plane

Items (Vehicle)

- **Dice**
- To view a **sub-cube** by selecting specific values for multiple dimensions.
- **Example:**
 - Select: Year = 2023, Location = Delhi & Mumbai, Product = Laptops
 - Result: Data only for those filters.
- Like cutting a smaller cube out of the big cube

- **Pivot:** It is also known as *rotation* operation as it rotates the current view to get a new view of the representation. In the sub-cube obtained after the slice operation, performing pivot operation gives a new view of it.

Pivot (Rotate)

To **rearrange** the dimensions for better visualization or understanding.

- **Example:**

- Swap rows and columns in a report: show **Products** on rows and **Years** on columns, or vice versa

Car				
Bus				
Train				
Plane				
	Delhi	Kolkata	Istanbul	Karachi

Operation	Description	Example
Drill Down	More detail	Quarter → Month
Roll Up	Less detail	Month → Quarter
Slice	Filter one dimension	Year = 2023
Dice	Filter multiple dimensions	Year = 2023, City = Delhi
Pivot	Reorient view	Swap Product with Time axis

- **Online Transaction Processing (OLTP)** is a way of handling data where many small transactions are processed **quickly and accurately**—usually in **real time**.

Real-time execution:

1. OLTP systems are designed to handle many transactions **immediately as they happen**.
2. Example: When you transfer money using online banking, it updates your balance right away.

Short, atomic operations:

1. These are **small, quick actions** like inserting, updating, or deleting a row in a database.
2. Each transaction is **atomic**, meaning it must be **fully completed or not happen at all**—no half-finished actions.

ACID properties:

OLTP systems follow these 4 rules to keep data reliable and accurate:

1. **Atomicity** – all parts of a transaction must succeed or fail together.
2. **Consistency** – the database must remain valid before and after the transaction.
3. **Isolation** – multiple transactions can happen at the same time without interfering with each other.
4. **Durability** – once a transaction is done, it stays done—even if the system crashes.

Used in critical systems:

1. OLTP is used in **everyday systems** that must be fast and reliable, such as:
 1. Online banking
 2. Airline reservation systems
 3. E-commerce checkout
 4. Hospital systems

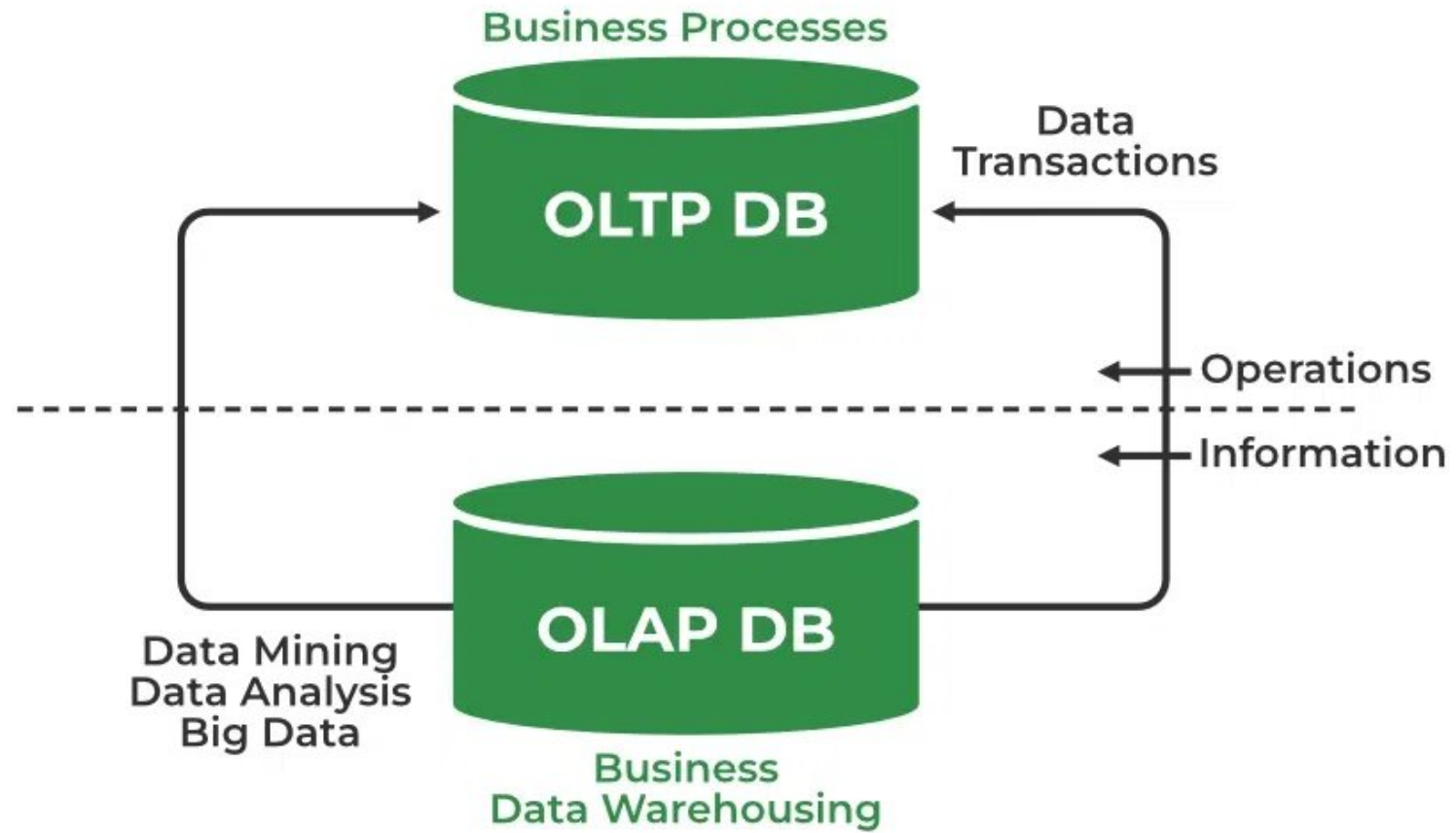
- OLTP focuses on handling large numbers of transactional operations in **real time**, ensuring data consistency and reliability for daily business operations.

- **Benefits of OLAP Services**

- Helps in keeping consistency and performing calculation on data.
- Can store planning, analysis, and budgeting for business analytics within one platform.
- Efficiently handle large volumes of data, making them suitable for enterprise-level business applications.
- Assist in applying security restrictions for data protection.
- Provide a multidimensional view of data, which helps in applying operations on data in various ways

- **OLTP Examples**

- An example considered for OLTP System is ATM Center a person who authenticates first will receive the amount first and the condition is that the amount to be withdrawn must be present in the ATM. The uses of the OLTP System are described below.
- ATM center is an OLTP application.
- OLTP handles the ACID properties during data transactions via the application.
- It's also used for Online banking, Online airline ticket booking, sending a text message, add a book to the shopping cart.



- **Online Transaction Processing (OLTP)**

- Online Transaction Processing, commonly known as OLTP, is a data processing approach emphasizing real-time execution of transactions. The majority of OLTP systems are meant to manage numerous short atomic operations that keep databases in line. To maintain transaction integrity and reliability, these systems support ACID (Atomicity, Consistency, Isolation, Durability) properties. It is through this that numerous unavoidable applications run their critical courses like online banking, reservation systems etc.

- **Benefits of OLTP Services**

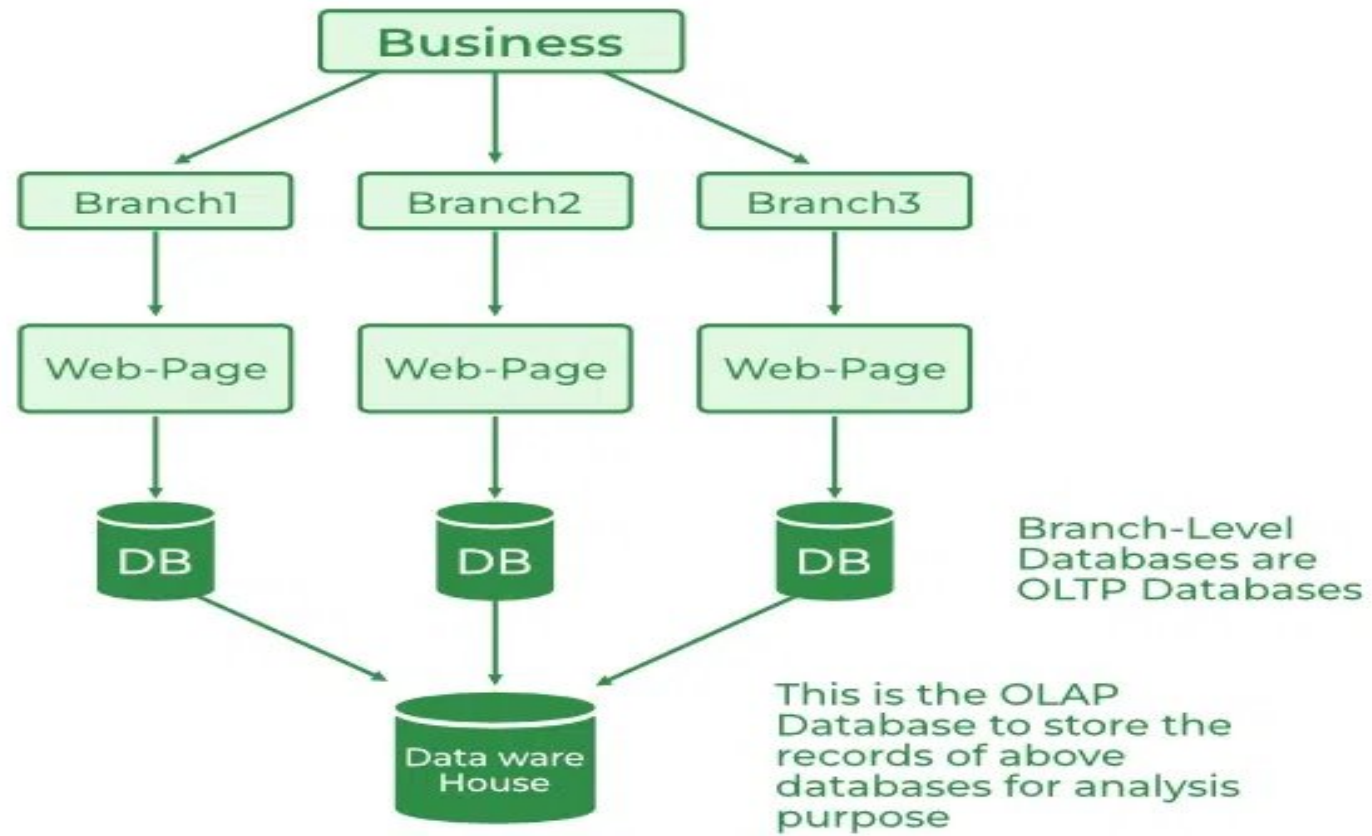
- Allow users to quickly read, write, and delete data operations.
- Support an increase in users and transactions for real-time data access.
- Provide better data protection through multiple security features.
- Aid in decision-making with accurate, up-to-date data.
- Ensure data integrity, consistency, and high availability.

- **Drawbacks of OLTP Services**

- Limited analysis capability, not suited for complex analysis or reporting.
- High maintenance costs due to frequent updates, backups, and recovery.
- Susceptible to disruption during hardware failures, impacting online transactions.
- Prone to issues like duplicate or inconsistent data.

OLTP and OLAP

- OLAP (Online Analytical Processing) and OLTP (Online Transaction Processing) are interconnected because they serve different but complementary roles in managing and using data within an organization:
- **OLTP** systems handle daily transactional data like sales, orders, or bookings. They are optimized for fast, real-time transaction processing and data entry.
- **OLAP** systems use the data collected by OLTP systems to perform complex analysis, reporting, and decision-making. OLAP organizes and summarizes this data to help identify trends and patterns.
- In short, OLTP captures and stores detailed transactional data, while OLAP processes that data for deeper insights, making OLTP the source of data for OLAP.



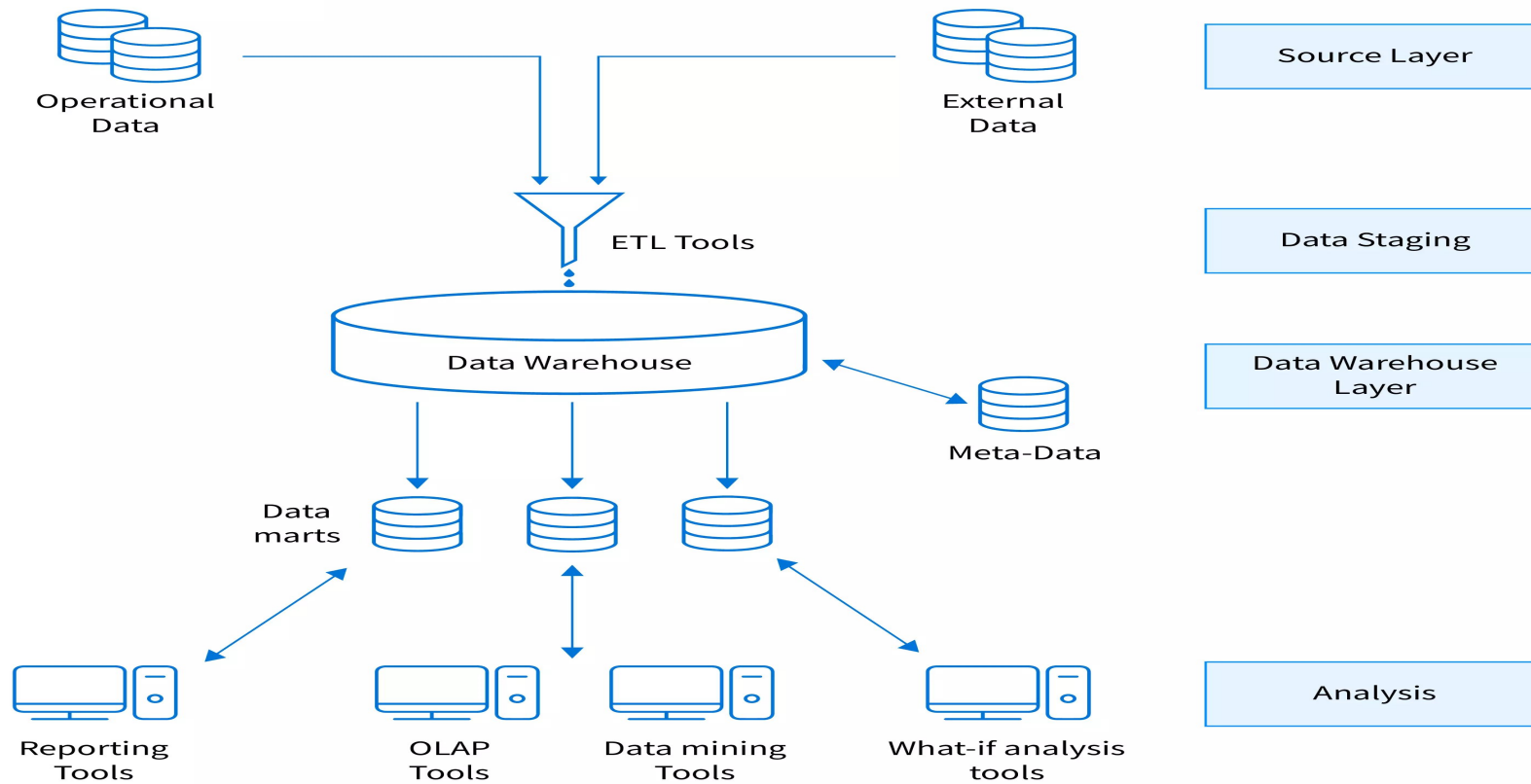
OLTP VS OLAP

Feature	OLTP (Online Transaction Processing)	OLAP (Online Analytical Processing)
Focus	Real-time transaction processing, day-to-day operations	Analyzing historical data for insights and decision-making
Data Volume	Smaller, focused on individual transactions	Large, historical datasets
Data Structure	Normalized, 3rd normal form, efficient for updates and insertions	Denormalized, Star or Snowflake schema, optimized for complex queries
Queries	Short, simple, CRUD operations (Create, Read, Update, Delete)	Long, complex, analytical queries involving aggregations and joins
Response Time	Fast, sub-millisecond for individual transactions	Slower, but optimized for large datasets
Examples	Banking transactions, e-commerce purchases, order processing	Sales analysis, customer behavior analysis, financial reporting

- **Two-Tier Data Warehouse Architecture**

- The single-tier architecture had a drawback because it didn't separate analytical and transactional processing. To fix this, the two-tier architecture was created, which separates these functions into different layers. It consists of two main parts: the Data Tier, where data is stored and processed, and the Client Tier, where users access and analyze the data. This separation improves efficiency and organization.

Two-Tier Data Warehouse Architecture



- In a two-tier data warehouse architecture, the **Data Tier** is where data is stored after it goes through the **ETL (Extract, Transform, Load)** process. This tier includes a **staging area** that collects and cleans data from multiple sources before storing it in a consistent format in the warehouse. The Data Tier has **three layers**: the **Source Layer**, the **Data Staging Layer**, and the **Data Warehouse Layer**.

Source Layer

- Source Layer is the first layer in the two-tier data warehouse architecture. It collects and stores data from multiple sources, such as corporate databases, legacy systems, and external sources. This layer serves as the starting point for further processing and analysis.

The Data Staging Layer

- The Data Staging Layer is the second layer in the two-tier data warehouse architecture. It takes raw data from the source layer and processes it through ETL (Extraction, Transformation, and Loading). This involves cleaning the data, fixing inconsistencies, and combining it into a common format. The goal is to prepare the data for storage and analysis.

- The **Data Warehouse Layer** is the third layer of the two-tier architecture. It stores the cleaned and transformed data from the staging layer in a central repository called the data warehouse.
- This data can also be used to create **data marts** for specific departments.
- Metadata repositories are included to manage details about sources, staging, and users. After this, the Client Tier is used to analyze and interpret the stored data.
-

- **The Client Tier**

The Client Tier is the front-end layer of the two-tier data warehouse architecture where users access and analyze clean data stored in the warehouse. It helps generate reports and insights to support decision-making and improve products or solutions. This tier includes the Analysis Layer, which presents data through user-friendly interfaces or dashboards for easy understanding.

The Analysis layer:

- The fourth layer of the two-tier data warehouse focuses on integrating data to enable flexible reporting, scenario simulation, and dynamic analysis. It includes tools like query optimizers and information navigators to help users explore data efficiently. However, this architecture has limitations, such as poor scalability, limited support for many users, and network connectivity issues.